UNIT 6

Q1) Explain the data structures required for two PASS Assembler in detail.

A two-pass assembler is a program that translates an assembly language source code into machine code (object code). It processes the source code in two distinct passes:

1. **First Pass**: The assembler reads through the source code to collect information about the labels (symbols) used in the program and generates an intermediate code, typically a symbol table.
2. **Second Pass**: The assembler translates the intermediate code into the actual machine code, replacing the symbolic addresses with the actual memory addresses obtained from the first pass.

To perform these operations efficiently, the assembler uses various data structures. The most important ones are the Symbol Table, Literal Table, and Opcode Table.

1. Symbol Table

The **Symbol Table** is a data structure used to store information about labels (also called symbols) in the program. In assembly language, labels are used to represent addresses or memory locations that are referenced throughout the program. During the first pass, the assembler collects these labels and their corresponding memory addresses.

Structure:

Symbol Name: The label or identifier in the program.
Memory Address: The actual memory address or location where the label will be placed after the second pass.
Type of Symbol: This field determines whether the symbol is a variable, constant, or a procedure.
Size: The size of the variable or procedure (if applicable).
Scope: Indicates whether the symbol is local to a function or global.

Working:

During the **first pass**, the assembler processes each instruction in the source code, identifying labels.
Each label is entered into the symbol table with a placeholder address.
During the **second pass**, the assembler replaces the labels with actual memory addresses.

Example:

Symbol	Memory Address	Type
`LOOP`	1000	Label
`A`	2000	Variable
`SUM`	3000	Procedure

2. Literal Table

The **Literal Table** stores all the literal values (constants) used in the program. Literals are values that are directly embedded in the program, such as numbers or string constants. These are usually initialized at specific addresses.

Structure:

Literal: The actual literal value (e.g., 100, "Hello", etc.).
Memory Address: The memory address where the literal will be stored in memory.
Type: This indicates whether the literal is a constant, string, or other types.

Working:

During the **first pass**, literals are identified and stored in the literal table with placeholder addresses.
In the **second pass**, the assembler resolves the actual address of the literal and places it in the final object code.

Example:

Literal	Memory Address	Type
`100`	5000	Integer
`"Hello"`	6000	String

3. Opcode Table

The **Opcode Table** contains the machine codes (binary or hexadecimal) corresponding to the instructions

(opcodes) in the assembly language. Each opcode in the assembly source code is mapped to a machine language instruction in this table.

Structure:

Instruction: The assembly instruction (e.g., MOV, ADD, SUB).

Opcode: The corresponding machine code (binary or hexadecimal).

Operand Type: The type of operands the instruction expects (e.g., register, immediate, memory address).

Working:

During both passes, the assembler looks up each instruction in the **opcode table** to get the corresponding machine code for that instruction.

In the **second pass**, the assembler replaces the mnemonic instructions with their corresponding opcodes.

Example:

Instruction	Opcode	Operand Type
`MOV`	`0001`	Register
`ADD`	`0010`	Immediate
`SUB`	`0011`	Memory

4. Data Structure for Intermediate Code Generation

In addition to the above tables, intermediate code generation might involve using a data structure like an **intermediate code table** (IC) to store the output of the first pass before the second pass. This table temporarily stores machine instructions that are generated during the first pass but are not yet finalized.

Working:

The **first pass** generates intermediate code, which may include symbolic addresses, literals, and uncompleted machine instructions.

The **second pass** completes these instructions, replacing symbols with their actual addresses.

Example:

Instruction	Operand 1	Operand 2
`MOV`	`A`	`100`
`ADD`	`B`	`200`

Q2) What is Macro? Explain macro call and macro expansion with suitable example.

A **macro** is a powerful feature in assembly languages and high-level programming languages that allows you to define a sequence of instructions or code fragments, which can be reused throughout the program. The sequence of code is given a name, and whenever the macro is called, the sequence is inserted at the point of the call. Macros help to avoid repetition and improve code readability and maintainability.

In assembly language, macros are often used to simplify complex operations or to provide functionality that might otherwise require multiple lines of code. Macros can take parameters, making them flexible and reusable in different contexts.

Macro Call and Macro Expansion

Macro Call:

A **macro call** is the point in the program where the macro is invoked. When a macro is called, its name is used like a function or command. The assembler then replaces the macro call with the code sequence (macro body) defined for that macro.

A macro call can take arguments (parameters), which can be used to make the macro more dynamic.

Macro Expansion:

- Macro expansion is the process where the macro call is replaced by the actual sequence of instructions (or code) defined in the macro body. This happens during the assembly process, specifically during the first pass.

- The assembler reads the macro definition and replaces every occurrence of the macro call with the corresponding code.

- If the macro has parameters, those parameters are substituted with the arguments passed in the macro call.

Let’s understand macros with an example in assembly language.

Macro Definition:

```
MACRO ADD_NUMS NUM1, NUM2
    MOV AX, NUM1        ; Load NUM1 into register AX
    ADD AX, NUM2        ; Add NUM2 to AX
ENDM
```

Here, ADD_NUMS is the macro name, and NUM1 and NUM2 are the parameters. This macro will add two numbers, NUM1 and NUM2, and store the result in the AX register.

Macro Call:

```
ADD_NUMS 5, 10      ; Call the macro with NUM1 = 5 and NUM2 = 10
```

This macro call will invoke the ADD_NUMS macro, passing 5 as NUM1 and 10 as NUM2.

Macro Expansion (After expansion, the code will look like this):

```
MOV AX, 5           ; Load 5 into AX register
ADD AX, 10          ; Add 10 to AX
```

Advantages of Macros:

Code Reusability: Macros help in reusing the same block of code multiple times, improving code readability and reducing redundancy.

Efficiency: Since macros are replaced by the assembler with the actual code, there is no function call overhead, making macros faster.

Customization: Macros can take parameters, allowing for more flexible and customizable code.

Disadvantages of Macros:

No Type Checking: The parameters passed to macros are not type-checked, so if incorrect types are passed, it may lead to errors during the program execution.

Debugging Difficulty: Since macros are expanded at the time of assembly, debugging them can be harder as the error messages may not indicate the source of the issue directly.

Q3) Forward Reference Problem in Assembly Programming

The **Forward Reference Problem** refers to a situation in assembly language (or similar low-level programming languages) where a label, variable, or symbol is used before it is defined. This issue arises because assemblers or compilers typically process code from top to bottom, and if a label or variable is referenced before it's declared or defined, the assembler cannot resolve the reference.

Why Does the Forward Reference Problem Occur?

In assembly programming, labels are often used to mark locations in the code, and they are referenced in various instructions, such as jumps, branches, or data manipulations. Assemblers process the code in a single pass, which means the code is typically read sequentially. If a label or variable is referenced before it is defined in the code, the assembler will not know its location, leading to errors.

Working of Forward Reference Problem:

- The assembler starts reading the source code from top to bottom.
- When it encounters a label reference, it expects the label to have been defined earlier or at least be declared.
- If the label is referenced before being defined, the assembler doesn't know where to go in the program, causing a **forward reference problem**.
- The issue prevents the program from being assembled correctly, leading to errors or unexpected behavior.

Example

Incorrect Example (Forward Reference Problem):

```
MOV AX, LABEL1      ; Reference to LABEL1 before it's defined
LABEL1:  DB 10      ; Label1 is defined here
```

Explanation of the issue:

- In the above example, the MOV AX, LABEL1 instruction refers to LABEL1 before it is defined.
- Since LABEL1 is defined later in the program, the assembler doesn't know what LABEL1 represents when it processes the MOV instruction.
- This results in a **forward reference problem** because the assembler encounters a reference before the actual definition of LABEL1.

Corrected Example (Resolved Forward Reference Problem):

```
LABEL1:  DB 10      ; LABEL1 is defined first

MOV AX, LABEL1      ; Now the reference to LABEL1 is valid
```

Explanation of the solution:

- In this corrected example, the label LABEL1 is defined before the MOV instruction.
- Now, when the assembler encounters MOV AX, LABEL1, it already knows what LABEL1 refers to, and it can properly assemble the code without errors.

Solutions to Forward Reference Problem:

1. Reordering Code: The simplest solution is to reorder the code so that labels, variables, or macros are defined before they are referenced. This ensures that by the time the assembler or preprocessor encounters the reference, it already knows the definition.

2. Use of Forward Declaration: Some assemblers support forward declarations or "prototypes" for labels or macros. This means you can inform the assembler about the symbol before it is defined, which allows the assembler to resolve the reference during subsequent passes.

Q4) What is Loader? What are the basic functions of loaders?

A **loader** is a system program that is responsible for loading an executable file into memory for execution. It performs the task of preparing the program code (machine code or object code) for execution by placing it in memory and handling various aspects of memory management and linking. The loader takes the compiled program (which may consist of multiple object files or modules) and loads it into memory, ready for execution.

In simple terms, a loader is responsible for transferring an executable program from secondary storage (like a disk) into main memory (RAM) and then starting its execution.

Basic Functions of a Loader:

1. **Loading the Program into Memory**:

The loader's primary function is to load the executable file into memory. It reads the compiled program (object code or machine code) from the storage device and loads it into the appropriate location in the memory.

Example: The loader may load the compiled code into a specific memory location defined by the operating system or based on the program's requirements (e.g., data segment, code segment).

2.**Memory Allocation**:

The loader allocates space in memory for the program's data and code segments. It ensures that sufficient memory is available to run the program and assigns addresses for each part of the program, such as global variables, code, and other data structures.

Example: The code segment (instructions) might be allocated to one part of memory, while the data segment (variables, arrays) might be allocated to another.

3.**Relocation**:

Relocation refers to adjusting the memory addresses in the program's code so that it can execute correctly regardless of where it is loaded in memory.

The loader changes the absolute addresses (from the compiled code) into relative addresses, so the program can be loaded into any part of memory, not necessarily where it was originally compiled.

Example: If the program is compiled to run at memory address 0x1000, but it's loaded into memory at address 0x2000, the loader will adjust all memory references (like variable addresses or jump instructions) accordingly.

4.**Symbol Resolution and Linking**:

In the case of a multi-module or multi-file program, the loader resolves symbols (variables, functions, etc.) that are defined in one module but referenced in another.

The loader combines object code from different modules and ensures that references between them are properly linked. This is typically done using **dynamic linking** where function calls are resolved during runtime.

Example: If a program calls a function defined in another object file, the loader will update the call to the correct memory address where the function resides.

5. Loading Shared Libraries (Dynamic Linking):

- For programs that rely on shared libraries (e.g., .dll files in Windows, .so files in Linux), the loader dynamically loads these libraries into memory during program execution.
- It also resolves references to functions or variables in these libraries, ensuring that the program can access

and call the functions it needs.
Example: If a program uses a standard library function like printf, the loader will ensure that the correct library (libc.so) is loaded and the function is resolved during runtime.
6. **Starting the Execution**:
Once the program is loaded into memory and all the references are resolved, the loader passes control to the program's entry point (typically the main() function in C or C++) and begins program execution.
Example: The loader will transfer control to the entry point of the program, which is where the program begins its execution.
7. **Error Handling**:
The loader is also responsible for handling errors related to loading, such as when the program cannot be loaded due to insufficient memory or if there are missing libraries or unresolved symbols.
Example: If the loader cannot find a required shared library, it may generate an error or report the missing dependency.
Types of Loaders:
1. Absolute Loader: Loads the program at a fixed memory location, and no relocation or dynamic linking is done. It is a simple loader, usually used in systems where memory allocation is static.
2. **Relocating Loader**: This loader supports relocating the program code to different memory locations. It adjusts memory addresses to match the location at which the program is loaded.
3. **Dynamic Loader**: This loader loads programs at runtime, resolving addresses dynamically, often used in systems that support dynamic linking. This type of loader allows programs to link to shared libraries at runtime.
4. **Bootstrap Loader**: A special loader that loads the operating system (OS) kernel into memory when the computer is powered on. It initializes the system and loads the OS to get the system up and running.
Example of Loader Working: Let's take an example of how a loader works in a typical system:
1. **Compilation and Linking**:
The source code (main.c, utils.c) is compiled into object files (main.o, utils.o) by the compiler.
A linker combines these object files into an executable file (program.exe), which contains machine code.
2. **Execution**:
The user runs the executable program.exe.
The loader loads the program.exe file into memory, allocating space for the program's code and data.
3. **Relocation**:
If the program is loaded into a memory location other than the one specified during compilation, the loader adjusts the memory addresses accordingly, so that all references (like variables and function calls) are valid.
4. **Symbol Resolution**:
If the program uses external functions from shared libraries, the loader resolves these function calls by loading the appropriate shared libraries (e.g., libc.so).
5. **Start Execution**:
The loader starts the program by transferring control to the program's entry point, such as the main() function, and the program starts executing.

Q5) What is system software? Explain any four system software in brief
System software refers to a collection of programs that manage and control the hardware components of a computer system and provide a platform for running application software. Unlike application software that is designed for end-users to perform specific tasks, system software is designed to facilitate the operation of the computer hardware and to provide the necessary environment for running application software.

The primary function of system software is to serve as an intermediary between hardware and application software. It includes essential programs that control and manage hardware, file systems, and other resources of the computer.
Four Types of System Software:
1. **Operating System (OS)**:
Description: The operating system is the most important system software. It acts as an interface between the computer hardware and the application software. The OS manages hardware resources like the CPU, memory, and I/O devices, providing services such as multitasking, file management, and device control.

Examples: Windows, Linux, macOS.
Key Functions:
Manages processes, memory, and file systems.
Controls hardware devices via drivers.
Provides security and user interface.
Example: When a user runs a program on a computer, the OS ensures that the program has access to memory, CPU, and other resources it needs to execute.
2. **Compiler**:
Description: A compiler is a system software tool that translates high-level programming language code (such as C, C++, or Java) into machine language code that the computer's CPU can execute. It takes the source code written by programmers and converts it into an executable file.
Examples: GCC, Java Compiler.
Key Functions:
Translates high-level programming language into machine code or intermediate code.
Helps in error detection and debugging.
Optimizes code for better performance.
Example: When writing a program in C, the developer uses a compiler to turn the C code into an executable binary file, which can be run on a computer.
3. **Device Drivers**:
Description: Device drivers are specialized software programs that allow the operating system and applications to interact with hardware devices. Each hardware device (such as printers, graphics cards, or network adapters) requires a corresponding driver to communicate with the OS and enable its functionality.
Examples: Printer drivers, graphics card drivers, sound card drivers.
Key Functions:
Enable communication between the OS and hardware.
Provide instructions for hardware operation.
Handle device-specific tasks like printing or rendering images.
Example: When a user connects a printer to a computer, the printer driver allows the OS to send print jobs to the printer and handle paper feed, ink levels, etc.
4. **Utility Programs**:
Description: Utility programs are small, specialized programs that help manage and tune the system's performance. These programs are designed to perform tasks related to system maintenance and optimization, such as managing files, diagnosing problems, and ensuring security.
Examples: Antivirus software, disk defragmenters, backup utilities.
Key Functions:
Maintain the health of the system by cleaning, optimizing, and protecting files.
Perform routine tasks like virus scanning, system backup, and file recovery.
Help in troubleshooting and fixing common system issues.
Example: An antivirus program scans for and removes malicious software, preventing security breaches on the system.

Q6) Explain imperative statement, declarative statement, and assembly directive
Assembly language programming uses different types of statements to control the flow of execution, allocate memory, and define data. The three types of statements are:
1. Imperative Statement:
Definition: Imperative statements describe the operations or instructions that need to be performed by the computer. These are instructions that actually perform actions, such as moving data, arithmetic operations, or branching.
Working:
An imperative statement directly modifies registers or memory locations, performs calculations, or controls program execution.
The CPU executes these instructions in the sequence specified.

Example:
```
MOV AX, 5       ; Move the value 5 into register AX
ADD AX, BX      ; Add the contents of register BX to AX
```
Explanation: In the above example, MOV and ADD are imperative statements that tell the CPU what actions to perform.

2. Declarative Statement:
Definition: Declarative statements do not perform operations but are used to define and declare data, constants, and labels in memory. These help in organizing the program by providing data structures and labels.
Working:
Declarative statements allocate memory or define constants and labels, enabling proper organization of data in the program.
They help the assembler understand what data structures are being used and where to store the information.
Example:
```
DATA SEGMENT
    NUM DB 10    ; Declare a byte variable NUM with initial value 10
DATA ENDS
```
Explanation: Here, NUM DB 10 is a declarative statement that allocates a byte of memory and initializes it with the value 10.

3. Assembly Directive:
Definition: Assembly directives are special instructions that provide information to the assembler but do not correspond to actual machine code instructions. These help control the assembly process, such as setting memory locations, defining data segments, and more.
Working:
Assembly directives provide the assembler with metadata, such as specifying where a program or data starts in memory, defining constants, or indicating where to store literals.
These are not part of the executable code but guide the assembly process.
Examples:
ORG (Origin): Specifies the starting address of the program or data in memory.
DB (Define Byte): Defines a byte of data.
DW (Define Word): Defines a word of data.
EQU (Equal): Defines a constant value.
Example:
```
ORG 1000H        ; Set the origin address to 1000H
MOV AX, NUM      ; Move the value of NUM into AX
NUM DB 10        ; Declare a variable NUM with value 10
```
Explanation: The directive ORG 1000H sets the starting address of the program to 1000H in memory.

Q7) Differentiate Between Literal and Immediate Operand [5 Marks]

Attribute	Literal Operand	Immediate Operand
Definition	A literal operand is a constant value that is embedded in the code, often stored in a literal pool.	An immediate operand is a constant value used directly in an instruction, typically within the instruction itself.
Memory Storage	Stored in a separate memory location called the literal pool.	Not stored separately; directly encoded in the instruction.
Example	`MOV AX, #5` (5 is a literal operand stored in memory)	`MOV AX, 5` (5 is an immediate operand used directly in the instruction)
Usage	Used when the same value needs to be referenced in multiple places in a program.	Used for quick, small, and temporary values directly in an instruction.
Location	Resides in the literal pool and is referenced by an address.	Part of the instruction encoding, usually within the instruction itself.

Q8) Explain ORIGIN, EQU, and LTORG with an Example

In assembly language programming, **ORIGIN**, **EQU**, and **LTORG** are important directives that help control the assembly process. These directives guide the assembler in managing memory, defining constants, and handling literals. Let's discuss each of them in detail, along with examples:

1. ORIGIN Directive (ORG)

The **ORIGIN (ORG)** directive is used to specify the starting address (location) in memory where the program or a specific part of the program should be placed. It is used to set the initial address for the code or data section in the memory.

Working:

The **ORG** directive helps control memory allocation by assigning the starting address for the program or data segment.

If we want the program to start at a specific memory location, we use this directive to inform the assembler where to begin placing the instructions or data.

Example:

```
ORG 1000H   ; Set the starting address for the program to 1000H
MOV AX, 5   ; Move the value 5 into register AX
MOV BX, 10  ; Move the value 10 into register BX
```

Explanation:

- The directive **ORG 1000H** tells the assembler to begin placing the code starting at the memory address 1000H.
- After this, the instructions (MOV AX, 5, MOV BX, 10) are placed at that memory address, ensuring the program starts at 1000H in memory.

2. EQU Directive (Equate)

The **EQU** directive is used to define constants or symbolic names for numeric values. It creates an alias for a value, allowing for easier and more readable code. The value associated with the symbol cannot be changed during program execution.

Working:

The **EQU** directive assigns a constant value to a symbolic name, making it easier to refer to the value in different parts of the program. This is especially useful for frequently used values like memory addresses, buffer sizes, etc.

Example:

```
MAX EQU 100       ; Define constant MAX as 100
MOV AX, MAX       ; Move the value 100 into register AX
```

Explanation:

The directive **MAX EQU 100** defines a constant MAX with the value 100.

Later, instead of using the number 100 directly, we can use the symbol MAX, improving code readability and maintainability.

3. LTORG Directive (Literal Pool)

The **LTORG** directive is used to instruct the assembler to create a literal pool. A literal pool is a section in memory that stores constants or literals used in the program, typically for things like strings or immediate values. The assembler places these literals in a separate section to manage memory efficiently.

Working:

The **LTORG** directive ensures that all the literal values used in the program (that are not immediately resolved) are placed into a literal pool.

It helps separate literals from the actual code, making the program more efficient by keeping literals in a designated area.

Example:

```
MOV AX, 'A'     ; Move literal 'A' into AX
LTORG           ; Generate the literal pool
MOV BX, 'B'     ; Move literal 'B' into BX
```

Explanation:
- The first line **MOV AX, 'A'** uses the literal 'A', which may not be resolved immediately.
- The **LTORG** directive tells the assembler to place the literal 'A' into a separate pool of literals.
- The literal pool is created after the **LTORG** directive, and the assembler knows that it should allocate a memory space for the literal values used later in the program.

Q9) List down the phases of a compiler. Explain with suitable example.
A compiler is a program that translates high-level source code written in a programming language (like C, Java, etc.) into machine code or an intermediate code that can be executed by the computer. The compilation process involves several phases, each responsible for a specific task in transforming the source code into an executable program.

The common phases of a compiler are:
1. Analysis Phase
The **Analysis Phase** is responsible for analyzing the source program to extract useful information and create intermediate representations. It is primarily concerned with understanding the structure and meaning of the source code. This phase includes the following sub-phases:
a. Lexical Analysis
Function: Lexical analysis is the first step in the analysis phase. It scans the source code and converts it into tokens, which are the smallest units of meaning in the source code.
Output: Tokens such as keywords, operators, identifiers, constants, and punctuation marks.
Example: For the statement int x = 10;, the tokens could be:

```
[ int, x, =, 10, ; ]
```

b. Syntax Analysis (Parsing)
Function: Syntax analysis checks whether the sequence of tokens generated by the lexical analyzer adheres to the grammatical structure of the programming language. It builds a **Syntax Tree** or **Parse Tree** that represents the syntactic structure of the program.
Output: A **parse tree** that represents the structure of the code.
Example: For int x = 10;, the syntax tree would show that it's a variable declaration of type int, with the variable x assigned the value 10.

c. Semantic Analysis
Function: Semantic analysis ensures that the program's logic is correct. It checks for semantic errors, such as type mismatches, undefined variables, or incorrect operations (e.g., trying to add a string to an integer).
Output: A program representation that is logically correct (with error messages if semantic errors are found).
Example: In the expression int x = "Hello"; (where "Hello" is a string), semantic analysis would detect a type mismatch, as x is declared as an integer but assigned a string.

2. Synthesis Phase
The **Synthesis Phase** is concerned with generating the final output, which is typically machine code or intermediate code. This phase takes the information gathered during the analysis phase and synthesizes it into an executable form. It includes the following sub-phases:
a. Intermediate Code Generation
Function: This phase converts the syntax tree into an intermediate representation, which is easier to manipulate than the original source code. This intermediate code is typically machine-independent, making it easier to apply optimizations and generate code for different machine architectures.
Output: Intermediate code, which is usually lower-level and platform-independent.
Example: The statement x = a + b; might be translated into intermediate code like:

```
t1 = a + b

x = t1
```

b. Code Optimization
Function: Code optimization improves the intermediate code by making it more efficient. The goal is to reduce the program's execution time, memory usage, or other resources without altering its functionality. This can include eliminating redundant operations, simplifying expressions, or rearranging instructions to reduce bottlenecks.
Output: Optimized intermediate code.
Example: If we have:

```
t1 = a + b
t2 = a + b
```

The optimization phase might simplify it to:

```
t1 = a + b
t2 = t1
```

c. Code Generation
Function: Code generation translates the optimized intermediate code into machine code or assembly code that can be executed by the computer. This phase is highly dependent on the target architecture.
Output: Machine code or assembly code.
Example: The intermediate code:

```
t1 = a + b
t2 = t1
```

Could be converted into machine-level assembly instructions like:

```
MOV AX, [a]
ADD AX, [b]
MOV [t1], AX
MOV [t2], [t1]
```

Working Example
Consider the following simple C program:

```
int x = 10;
int y = x + 5;
```

Analysis Phase:
1. Lexical Analysis: Tokens: [int, x, =, 10, ;, int, y, =, x, +, 5, ;]
2. Syntax Analysis: Parse Tree: This tree shows that int x = 10; is a variable declaration, and y = x + 5; is an expression with a variable y being assigned the result of x + 5.
3. Semantic Analysis: Ensures that x and y are valid variables and that x + 5 is a valid expression. If there were an error (e.g., trying to assign a string to an integer), it would be caught here.

Synthesis Phase:
1. Intermediate Code Generation:
For y = x + 5;, the intermediate code could be:

```
t1 = x + 5
y = t1
```

2. Code Optimization: If there were redundant operations, the optimizer would remove them. In this case, no obvious optimization is needed.
3. Code Generation: The optimized intermediate code could be converted to assembly or machine code:

```
MOV AX, [x]
ADD AX, 5
MOV [y], AX
```